



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of Computing

Semester 2

Session 2025/2026

SECJ3483 - SECTION 01
WEB TECHNOLOGY

Group Lab Activity: Security Investigation and Fix

Project Title : Secure Person BMI Full-Stack Web Application

Name	Matrics No.
MUHAMMAD AIDIL HAIKAL BIN MAZALAN	A23CS5010
MUHAMMAD AIMAN DANISH BIN MUHAMMAD EKHSAN	A23CS0119
MUHAMMAD FAKHRUL RAZZI BIN MD NOOR	A23CS0128
MUHAMMAD RIFQI BIN RAZALI	A23CS0136

2. Security investigation summary

Test No	What I Tested	Expected Secure Behavior	Actual Result	Is It a Weakness?	Why?
1	Add BMI with invalid data	Backend should reject invalid data	Data is accepted and stored in the database.	Yes	Missing backend validation because frontend validation can be bypassed using ThunderClient.
2	Access another user's BMI record	Backend should deny access	The private BMI record data is returned.	Yes	Broken Object Level Authorization because data ownership is not verified after a user logs in.
3	Access staff route as normal user	Backend should deny access	The user is allowed to access and view the data.	Yes	Broken Function Level Authorization because there are no server-side role checks for staff features.
4	Access admin route as normal user	Backend should deny access	The user is allowed to access and view the data.	Yes	Broken Function Level Authorization because the route lacks server-side admin protection middleware.
5	Submit XSS payload in notes	Payload should display as text	The browser executes the script by showing the alert.	Yes	Stored Cross-Site Scripting because the frontend renders input unsafely using v-html.
6	Check API response	Password/hash should not be visible	The password value is visible in the payload.	Yes	Sensitive data exposure because the API blindly returns all columns including password_hash.
7	Try SQL Injection input	Input should be treated as data	The structural database query behaves unexpectedly.	Yes	SQL Injection because the application constructs database queries using unsafe string concatenation.
8	Try to update protected fields	Backend should reject or ignore protected fields	The fields (e.g., role, user_id) are accepted and modified	Yes	Mass assignment because the backend blindly executes updates on protected fields like user_id and role.

Test 1: Add BMI with invalid data

- Did the backend accept the data?

Yes, it saved the negative and empty values directly into the database.

- Did the frontend validation stop you?

No, because I can bypass the frontend UI using Thunder Client.

- Can the same request be sent using Postman?

Yes, anyone can intercept and send raw requests to the API.

- Where should the validation be enforced?

Backend, because frontend validation can be bypassed.

The screenshot shows the Thunder Client interface with a POST request to `http://localhost:8080/api/persons`. The request body is a JSON object with invalid values: `"name": "", "age": -5, "height": 0, "weight": -70, "bmi": "0.00", "category": "", "notes": "testing invalid BMI"`. The response status is `201 Created` with a size of `691 Bytes` and a time of `44 ms`. The response body is a JSON object containing a message, the person data, and debug information including the received body and the SQL query executed.

```
1 {
2   "message": "BMI record created. This route trusts frontend data.",
3   "person": {
4     "id": 22,
5     "user_id": 1,
6     "name": "",
7     "age": -5,
8     "height": "0.00",
9     "weight": "-70.00",
10    "bmi": "0.00",
11    "category": "",
12    "notes": "testing invalid BMI",
13    "created_at": "2026-06-26 11:08:22"
14  },
15  "debug_received_body": {
16    "name": "",
17    "age": -5,
18    "height": 0,
19    "weight": -70,
20    "notes": "testing invalid BMI"
21  },
22  "debug_sql": "INSERT INTO persons (user_id, name, age, height, weight, bmi, category, notes)\n                VALUES (1, '', -5, 0, -70, 0, '', 'testing invalid BMI')"
```

Test 2: Access another user's BMI record

- Did the backend return another user's BMI record?

Yes, changing the ID in the URL leaked other people's data.

- Is login alone enough?

No, it is just authentication but no authorization.

- What should the backend compare?

It must match the `user_id` from the JWT with the `user_id` on the database.

- Should staff and admin have different access?

Yes, staff and admin roles need to have different access to monitor all records.

TC New Request X README.md U Insecure Person BMI App - Vue ... localhost:8080/api/persons Analyze files with PVS-Studio

GET http://localhost:8080/api/persons/1 Send

Query Headers 1 Auth Body Tests Pre Run

HTTP Headers ☐ Raw

☐ Accept	/*
☐ User-Agent	Thunder Client (https://www.thunderclient.
☒ Authorization	eyJ1c2VyX2lkjoxLCjyb2xlljoidXNlcislmVtY
☐ Content-Type	value
☐ header	value

Status: 200 OK Size: 435 Bytes Time: 46 ms

Response Headers 9 Cookies Results Docs </> ≡

```
1 {
2   "message": "Record returned without ownership check.",
3   "person": {
4     "id": 1,
5     "user_id": 1,
6     "name": "Muhammad Aiman Hakimi",
7     "age": 21,
8     "height": "1.70",
9     "weight": "65.00",
10    "bmi": "22.49",
11    "category": "Normal",
12    "notes": "Student sample BMI record",
13    "created_at": "2026-06-22 11:28:30"
14  },
15  "debug_sql": "SELECT * FROM persons WHERE id = 1"
16 }
```

TC New Request X README.md U Insecure Person BMI App - Vue ... localhost:8080/api/persons Analyze files with PVS-Studio

GET http://localhost:8080/api/persons/3 Send

Query Headers 1 Auth Body Tests Pre Run

HTTP Headers ☐ Raw

☐ Accept	/*
☐ User-Agent	Thunder Client (https://www.thunderclient.
☒ Authorization	eyJ1c2VyX2lkjoxLCjyb2xlljoidXNlcislmVtY
☐ Content-Type	value
☐ header	value

Status: 200 OK Size: 435 Bytes Time: 50 ms

Response Headers 9 Cookies Results Docs </> ≡

```
1 {
2   "message": "Record returned without ownership check.",
3   "person": {
4     "id": 3,
5     "user_id": 3,
6     "name": "Ahmad Farhan Roslan",
7     "age": 21,
8     "height": "1.75",
9     "weight": "82.00",
10    "bmi": "26.78",
11    "category": "Overweight",
12    "notes": "Needs weight monitoring",
13    "created_at": "2026-06-22 11:28:30"
14  },
15  "debug_sql": "SELECT * FROM persons WHERE id = 3"
16 }
```

Test 3: Access staff route as normal user

- Did the backend block the request?

No, a normal user token could access the restricted endpoints.

- Which roles should be allowed?

Only users with a role value of 'staff' or 'admin'.

- Should checking be done in frontend route guard only?

No, attackers don't use the frontend browser to make API requests.

- Where must the final decision happen?

In the backend routing middleware.

The screenshot shows a web client interface with the following details:

- Request:** GET `http://localhost:8080/api/staff/persons`
- Status:** 200 OK, Size: 9.7 KB, Time: 22 ms
- Response Body (JSON):**

```
{
  "message": "All BMI records returned without staff role check.",
  "persons": [
    {
      "id": 22,
      "user_id": 1,
      "name": "",
      "age": -5,
      "height": "0.00",
      "weight": "-70.00",
      "bmi": "0.00",
      "category": "",
      "notes": "testing invalid BMI",
      "created_at": "2026-06-26 11:08:22",
      "owner_email": "aiman@student.utm.my",
      "owner_role": "user"
    },
    {
      "id": 21,
      "user_id": 1,
      "name": "",
      "age": -5,
      "height": "0.00",
      "weight": "-70.00",
      "bmi": "999.00",
      "category": "Normal",
      "notes": "invalid test",
      "created_at": "2026-06-26 10:42:47",
      "owner_email": "aiman@student.utm.my",
      "owner_role": "user"
    },
    {
      "id": 20,
      "user_id": 20,
      "name": "Mazlina Ahmad",
      "age": 45,
      "height": "1.60",
      "weight": "58.00"
    }
  ]
}
```

Test 4: Access admin route as normal user

- Did the backend accept user_id?

Yes.

- Did the backend accept role?

Yes.

- Did the backend accept bmi and category from the frontend?

Yes, the database blindly updated them all.

- Which fields should be allowed?

Only name, age, height, weight, and notes.

- Which fields should be calculated or controlled by backend?

Structural fields like user_id and role, plus calculations like bmi and category.

The screenshot shows a web browser's developer tools with the 'Response' tab selected. The request is a GET to `http://localhost:8080/api/admin/users`. The response status is 200 OK, with a size of 5.8 KB and a time of 26 ms. The response body is a JSON array of four user objects. Each object contains fields for id, name, email, password, password_hash, role, and created_at. The roles for all users are 'user'.

```
1 {
2   "message": "All users returned without admin role check. Sensitive fields exposed.",
3   "users": [
4     {
5       "id": 1,
6       "name": "Muhammad Aiman Hakimi",
7       "email": "aiman@student.utm.my",
8       "password": "password123",
9       "password_hash": "password123",
10      "role": "user",
11      "created_at": "2026-06-22 11:28:30"
12    },
13    {
14      "id": 2,
15      "name": "Nur Aisyah Zulkifli",
16      "email": "aisyah@student.utm.my",
17      "password": "password123",
18      "password_hash": "password123",
19      "role": "user",
20      "created_at": "2026-06-22 11:28:30"
21    },
22    {
23      "id": 3,
24      "name": "Ahmad Farhan Roslan",
25      "email": "farhan@student.utm.my",
26      "password": "password123",
27      "password_hash": "password123",
28      "role": "user",
29      "created_at": "2026-06-22 11:28:30"
30    },
31    {
32      "id": 4,
33      "name": "Siti Nur Balqis Ismail",
34      "email": "balqis@student.utm.my",
35      "password": "password123",
36      "password_hash": "password123",
37      "role": "user",
38      "created_at": "2026-06-22 11:28:30"
39    }
40  ]
41 }
```

Test 5: Submit XSS payload in notes

- Does the API return password_hash?

Yes, it was fully visible in the JSON response payload.

- Does the frontend really need this data?

No, the client UI never needs to know password hashes.

- What fields should be returned?

Only safe profile data: id, name, email, and role.

- How can SQL SELECT be improved?

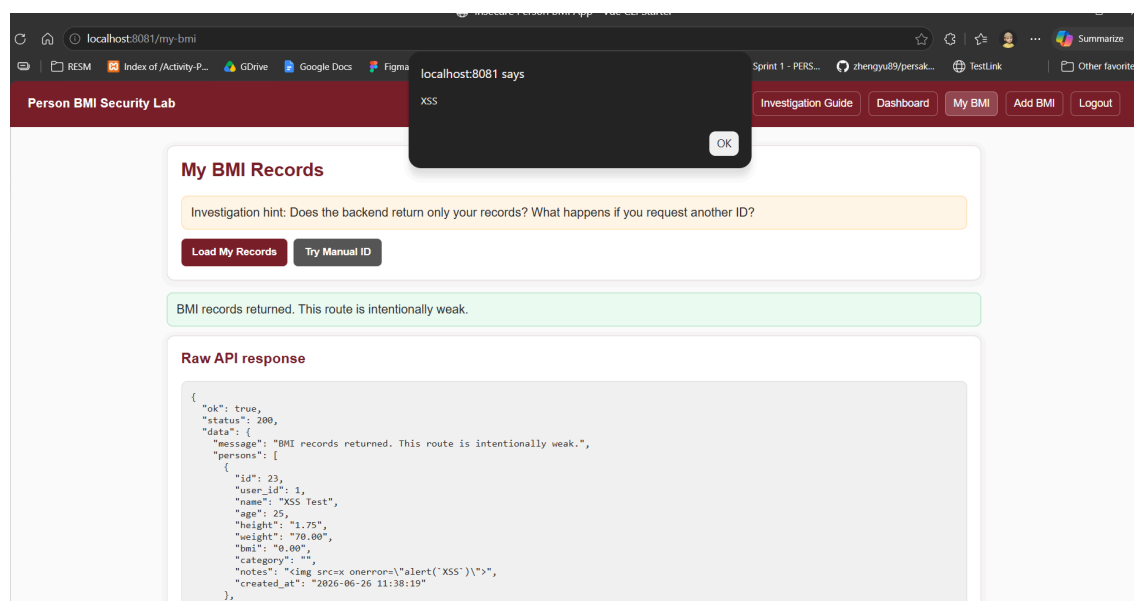
Stop using SELECT * and explicitly write out the safe columns.

```
POST http://localhost:8080/api/persons

{
  "name": "XSS Test",
  "age": 25,
  "height": 1.75,
  "weight": 70,
  "notes": "<img src=x onerror='alert('XSS')'\>"
}
```

```
201 Created  Size: 769 Bytes  Time: 47 ms

{
  "message": "BMI record created. This route trusts frontend data.",
  "person": {
    "id": 23,
    "user_id": 1,
    "name": "XSS Test",
    "age": 25,
    "height": 1.75,
    "weight": 70.00,
    "bmi": 0.00,
    "category": "",
    "notes": "<img src=x onerror='alert('XSS')'\>",
    "created_at": "2026-06-26 11:38:19"
  },
  "debug_received_body": {
    "name": "XSS Test",
    "age": 25,
    "height": 1.75,
    "weight": 70,
    "notes": "<img src=x onerror='alert('XSS')'\>"
  },
  "debug_sql": "INSERT INTO persons (user_id, name, age, height, weight, bmi, category, notes)\nVALUES (1, 'XSS Test', 25, 1.75, 70, 0, '', '<img src=x onerror='alert('XSS')'\>')"
```



Test 6: Check API response

- Did the alert execute?

Yes, the script payload popped up in the browser window.

- Is the frontend using innerHTML or v-html?

Yes, it used unsafe binding instead of text escaping.

- What is safer for displaying normal text?

Double curly braces {{ }} text interpolation or textContent.

- Should user-generated content be rendered as HTML?

No, never render raw user input as live HTML.

The screenshot displays the Thunder Client interface. The top bar shows the current request: GET http://localhost:8080/api/admin/users. The left sidebar contains the 'HTTP Headers' tab, which is active. It lists several headers: 'Accept' (*/*), 'User-Agent' (Thunder Client), 'Authorization' (eyJ1c2VyX2lkjoxLjYb2xlioidXNlciismVtYn), 'Content-Type' (application/json), and a generic 'header' (value). The right pane shows the 'Response' tab, displaying a JSON object. The status is 200 OK, with a size of 5.8 KB and a time of 39 ms. The JSON response includes a message and an array of user objects.

```
1 {
2   "message": "All users returned without admin role check. Sensitive fields exposed.",
3   "users": [
4     {
5       "id": 1,
6       "name": "Muhammad Aiman Hakimi",
7       "email": "aiman@student.utm.my",
8       "password": "password123",
9       "password_hash": "password123",
10      "role": "user",
11      "created_at": "2026-06-22 11:28:30"
12    },
13    {
14      "id": 2,
15      "name": "Nur Aisyah Zulkifli",
16      "email": "aisyah@student.utm.my",
17      "password": "password123",
18      "password_hash": "password123",
19      "role": "user",
20      "created_at": "2026-06-22 11:28:30"
21    },
22    {
23      "id": 3,
24      "name": "Ahmad Farhan Roslan",
25      "email": "farhan@student.utm.my",
26      "password": "password123",
27      "password_hash": "password123",
28      "role": "user",
29      "created_at": "2026-06-22 11:28:30"
30    },
31    {
32      "id": 4,
33      "name": "Siti Nur Balqis Ismail",
34      "email": "balqis@student.utm.my",
35      "password": "password123",
36      "password_hash": "password123",
37      "role": "user",
38      "created_at": "2026-06-22 11:28:30"
39    }
40  ]
41 }
```


Test 7: Try SQL Injection input

- Did the SQL query behave unexpectedly?

Yes, inputting quotes broke out of the string and changed the logic.

- Was login bypassed?

Yes, the payload successfully manipulated the query parameters.

- Was extra data returned?

Yes, it fetched records without validating real credentials.

- Is the SQL query using string concatenation?

Yes, it directly glued variables into the raw query string.

The screenshot shows a web browser's developer tools with the 'Network' tab selected. A POST request to `http://localhost:8080/api/login` is shown. The request body is a JSON object: `{ "email": "aiman@student.utm.my' -- ", "password": "blablalba" }`. The response status is 200 OK, with a size of 735 Bytes and a time of 49 ms. The response body is a JSON object containing a success message, a token, user details, and debug information. The debug information shows the raw SQL query that was executed: `"SELECT * FROM users WHERE email = 'aiman@student.utm.my' -- ' AND password = 'blablalba' LIMIT 1"`. This confirms that the SQL query was using string concatenation and that the injection was successful.

```
POST http://localhost:8080/api/login
{
  "email": "aiman@student.utm.my' -- ",
  "password": "blablalba"
}
```

```
{
  "message": "Login successful. This token is intentionally insecure.",
  "token": "eyJ1c2VyX2lkIjoxLCJyb2x1IjoidXNlciIsImVtYWlsIjoiYVltYV5Ac3R1ZGVudC51dG0ubXkiLCJub3R1IjoSU5STRUNVUKVfrkFLRV9UT0tFTl9OT19TSudoQVRVUkVfIk9frVhQSVZlIn0=",
  "user": {
    "id": 1,
    "name": "Muhammad Aiman Hakimi",
    "email": "aiman@student.utm.my",
    "password": "password123",
    "password_hash": "password123",
    "role": "user",
    "created_at": "2026-06-22 11:28:30"
  },
  "debug_received_body": {
    "email": "aiman@student.utm.my' -- ",
    "password": "blablalba"
  },
  "debug_sql": "SELECT * FROM users WHERE email = 'aiman@student.utm.my' -- ' AND password = 'blablalba' LIMIT 1"
}
```

Test 8: Try to update protected fields

The screenshot shows a REST client interface with a PUT request to `http://localhost:8080/api/persons/1`. The response status is 200 OK, with a size of 769 Bytes and a time of 57 ms. The response body is a JSON object containing a message, a person object, debug information, and an SQL query.

Request:

```
PUT http://localhost:8080/api/persons/1
```

Response:

```
1 {
2   "message": "BMI record updated. This route allows unsafe field
3   updates.",
4   "person": {
5     "id": 1,
6     "user_id": 2,
7     "name": "Ali Updated",
8     "age": 22,
9     "height": "1.70",
10    "weight": "65.00",
11    "bmi": "10.00",
12    "category": "Normal",
13    "notes": "Student sample BMI record",
14    "created_at": "2026-06-22 11:28:30"
15  },
16  "debug_received_body": {
17    "name": "Ali Updated",
18    "age": 22,
19    "height": 1.7,
20    "weight": 65,
21    "user_id": 2,
22    "role": "admin",
23    "bmi": 10,
24    "category": "Normal"
25  },
26  "debug_sql": "UPDATE persons SET user_id = 2, name = 'Ali Updated'
, age = 22, height = 1.7, weight = 65, bmi = 10, category =
'Normal' WHERE id = 1"
27 }
```

3. Weaknesses found

Weakness Found	Frontend / Backend / API / Database	Security Category	Possible Risk
Negative weight accepted	Backend	Missing backend validation	Invalid data stored
User accesses another BMI record	API / Backend	Broken Object Level Authorization	Privacy violation
Normal user accesses admin route	API / Backend	Broken Function Level Authorization	Unauthorized admin action
API returns password_hash	API / Backend	Sensitive data exposure	Credential risk
XSS payload executes	Frontend	XSS	Browser script execution
SQL Injection input affects query	Backend / Database	SQL Injection	Unauthorized data access
User changes user_id or role	API / Backend	Mass assignment	Privilege or ownership manipulation

Discussion Questions

1. Which weakness is the most dangerous? Why?
 - SQL Injection is among the most dangerous because it can allow attackers to bypass authentication, read, modify, or delete database data if queries are not protected.
2. Which weakness is caused by trusting the frontend?
 - Missing backend validation because attackers can bypass frontend validation using Postman or Thunder Client.
3. Which weakness is caused by missing backend validation?
 - Invalid BMI values such as negative weight, zero height, and empty names.
4. Which weakness is caused by missing authorization?
 - Broken Object Level Authorization and Broken Function Level Authorization. These allow users to access data or functions they should not have permission to use.
5. Which weakness can expose private data?
 - Sensitive Data Exposure, such as returning password_hash or allowing users to view another user's BMI records.
6. Which weakness can affect the browser?
 - Cross-Site Scripting (XSS), which can execute malicious JavaScript in the user's browser.
7. Which weakness can affect the database?
 - SQL Injection, because attackers may manipulate SQL queries to read, modify, or delete data.
8. Which weakness can be prevented by JWT?

- JWT helps verify a user's identity (authentication) and supports protecting routes from unauthenticated access.

9. Which weakness cannot be solved by JWT alone?

- SQL Injection
- XSS
- Backend validation issues
- Owner-based access control
- RBAC (roles still need to be checked after verifying the token)

10. Which fixes must be implemented in the backend?

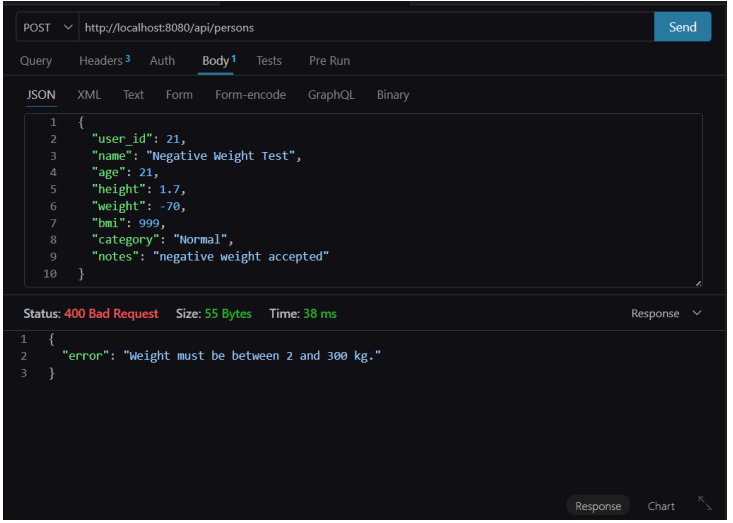
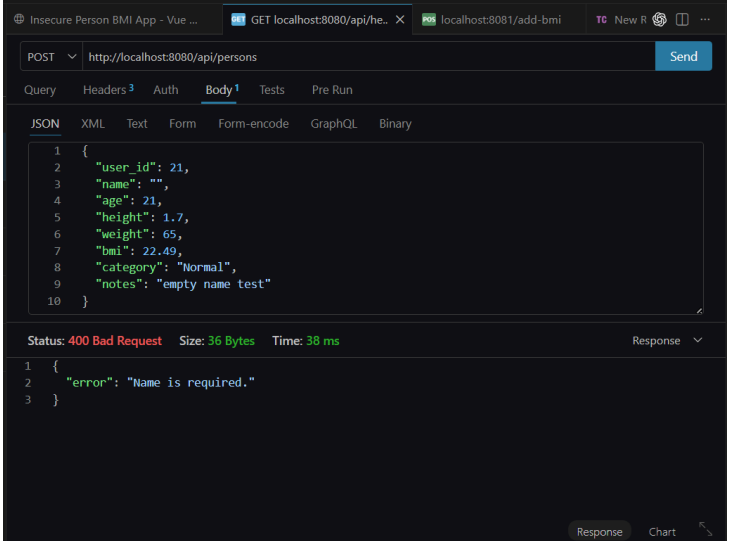
- Input validation
- BMI calculation
- Password hashing
- Prepared statements
- JWT verification
- RBAC checks
- Owner-based access control
- Safe API responses
- Preventing mass assignment

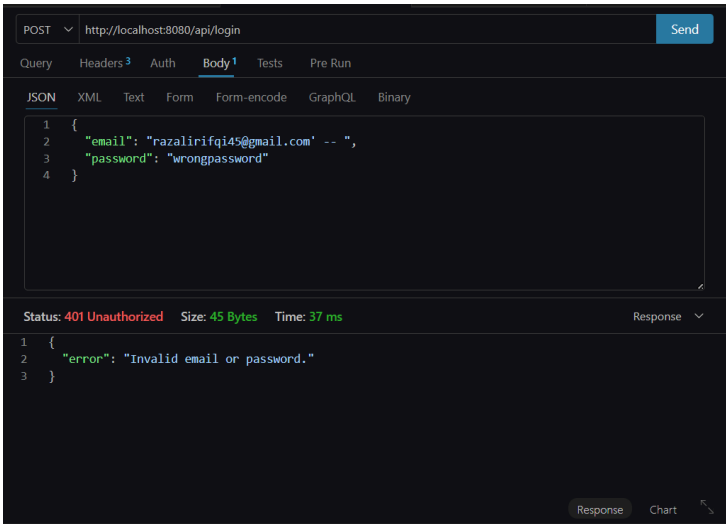
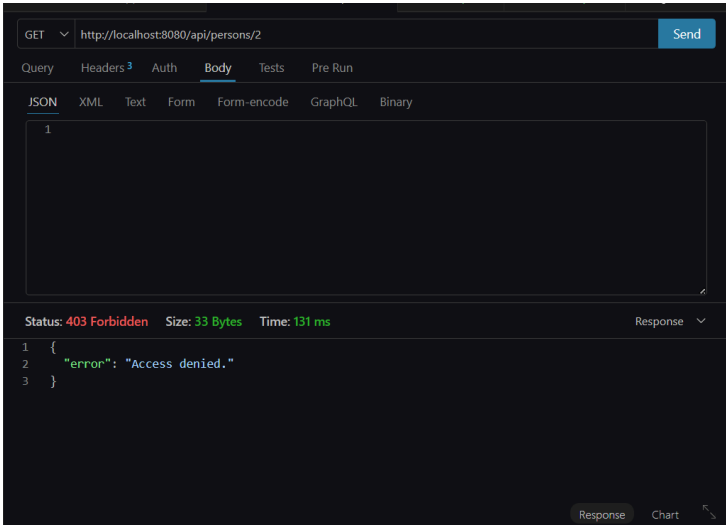
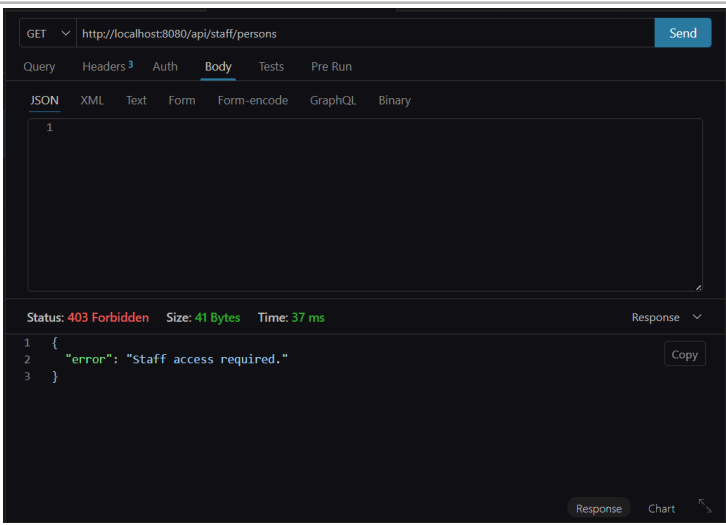
4. Fixes Implementation

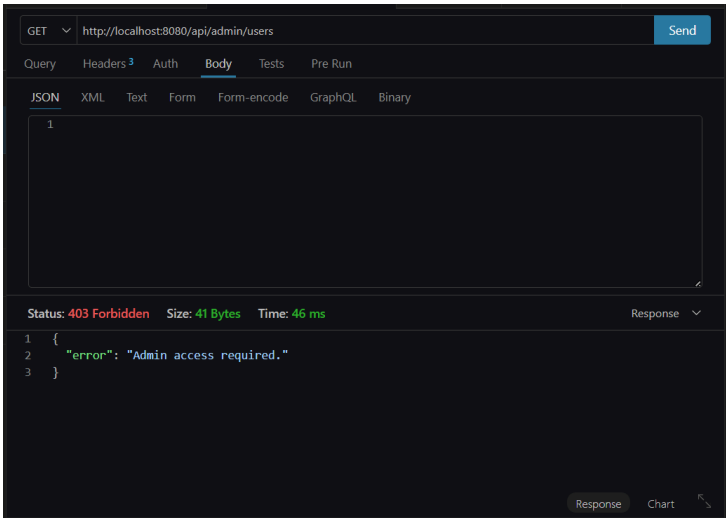
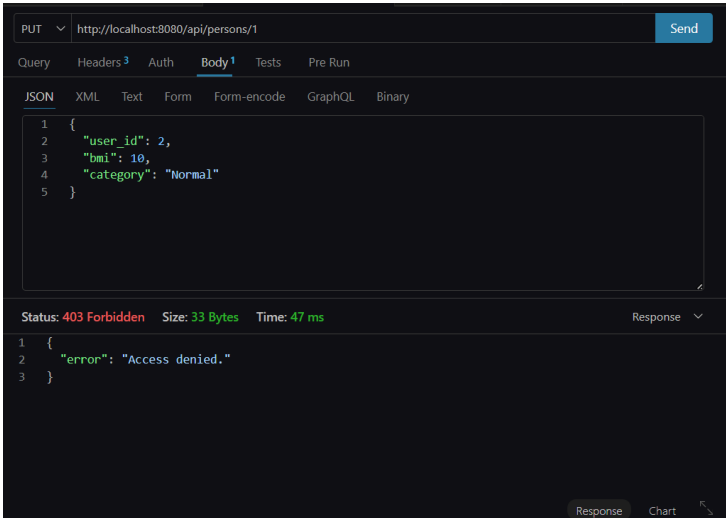
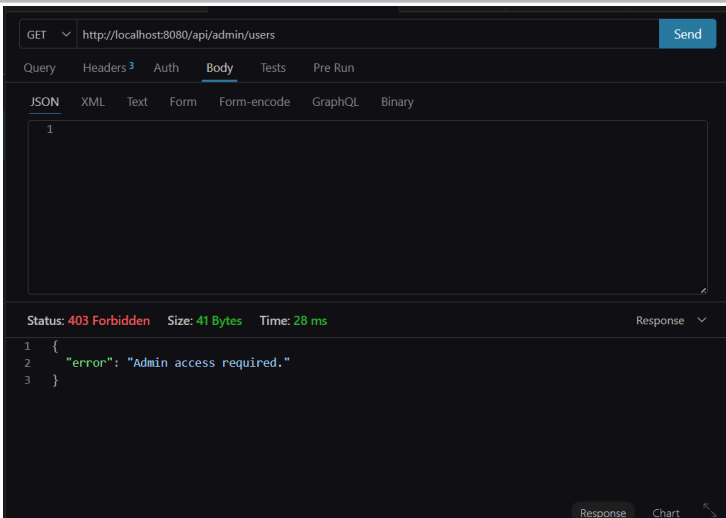
Problem	Root Cause	Where to Fix?	Proposed Solution	How to Retest?
Invalid BMI accepted	No backend validation	Backend	Validate name, age, height, and weight	Submit invalid data again
User accesses other record	No ownership check	Backend API	Compare JWT user_id with record user_id	Access another user's record
User accesses staff route	No role check	Backend API	Check role from JWT – staff or admin only	Use a user token on the staff route
User accesses admin route	No role check	Backend API	Check role from JWT – admin only	Use a user token on the admin route
XSS executes	Unsafe v-html rendering	Frontend	Replace v-html with {{ }} interpolation	Submit the XSS payload again
password_hash returned	SELECT * used	Backend API	Select only safe fields explicitly	Check the JSON response
SQL Injection possible	String concatenation in SQL	Backend	Use prepared statements	Try the injection input again
user_id can be changed	No field whitelist	Backend API	Allow only name, age, height, weight, and notes	Send user_id in the PUT request

5. Testing evidence

After Fix Implementation

Test Case	Evidence
Add BMI with negative weight	 <pre>POST http://localhost:8080/api/persons { "user_id": 21, "name": "Negative Weight Test", "age": 21, "height": 1.7, "weight": -70, "bmi": 999, "category": "Normal", "notes": "negative weight accepted" }</pre> Status: 400 Bad Request Size: 55 Bytes Time: 38 ms <pre>{ "error": "Weight must be between 2 and 300 kg." }</pre>
Submit empty name	 <pre>POST http://localhost:8080/api/persons { "user_id": 21, "name": "", "age": 21, "height": 1.7, "weight": 65, "bmi": 22.49, "category": "Normal", "notes": "empty name test" }</pre> Status: 400 Bad Request Size: 36 Bytes Time: 38 ms <pre>{ "error": "Name is required." }</pre>

Test Case	Evidence
SQL Injection login input	 POST http://localhost:8080/api/login Send Query Headers ³ Auth <u>Body</u> ¹ Tests Pre Run JSON XML Text Form Form-encode GraphQL Binary <pre>1 { 2 "email": "razalirifqi45@gmail.com" -- ", 3 "password": "wrongpassword" 4 }</pre> Status: 401 Unauthorized Size: 45 Bytes Time: 37 ms Response Chart <pre>1 { 2 "error": "Invalid email or password." 3 }</pre>
Access another user's BMI record	 GET http://localhost:8080/api/persons/2 Send Query Headers ³ Auth <u>Body</u> Tests Pre Run JSON XML Text Form Form-encode GraphQL Binary <pre>1</pre> Status: 403 Forbidden Size: 33 Bytes Time: 131 ms Response Chart <pre>1 { 2 "error": "Access denied." 3 }</pre>
Access staff route as user	 GET http://localhost:8080/api/staff/persons Send Query Headers ³ Auth <u>Body</u> Tests Pre Run JSON XML Text Form Form-encode GraphQL Binary <pre>1</pre> Status: 403 Forbidden Size: 41 Bytes Time: 37 ms Response Chart <pre>1 { 2 "error": "Staff access required." 3 }</pre>

Test Case	Evidence
Access admin route as user	 GET http://localhost:8080/api/admin/users Status: 403 Forbidden Size: 41 Bytes Time: 46 ms <pre>1 { 2 "error": "Admin access required." 3 }</pre>
Update user_id or role	 PUT http://localhost:8080/api/persons/1 Status: 403 Forbidden Size: 33 Bytes Time: 47 ms <pre>1 { 2 "error": "Access denied." 3 }</pre>
API returns password_hash	 GET http://localhost:8080/api/admin/users Status: 403 Forbidden Size: 41 Bytes Time: 28 ms <pre>1 { 2 "error": "Admin access required." 3 }</pre>

Test Case	Evidence
XSS payload in notes	Person BMI Security Lab Investigation Guide Dashboard My BMI Add BMI Logout Add BMI Record Insecure starter: BMI and category are calculated in frontend and sent to backend. Students should question this design. Name Age Height (meters) Weight (kg) All 21 1.70 65 Notes Frontend-calculated BMI Frontend-calculated Category 22.49 Normal ► Insecure payload preview Save BMI BMI record created. Raw API response XSS Test #23 Owner user_id: 1 Age: 21 Height: 1.70 m Weight: 65.00 kg BMI: 22.49 Category: Normal Notes: Delete Try ID + 1

6. GitHub commit summary

Commit	Commit Message	Assignee
Commit 1	Unzip the project and deploy to /vue/frontend and /htdocs/backend	Muhammad Aiman Danish bin Muhammad Ekhsan
	Initial security investigation	
Commit 2	Add validation password hashing and prepared statements	Muhammad Fakhru bin Md Noor
Commit 3	Add JWT authentication and protected routes	Muhammad Rifqi bin Razali
Commit 4	Add RBAC ownership checks and final security fixes	Muhammad Aidil Haikal bin Mazalan
Commit 5	Add Final Lab Report	

7. Reflection

1. Which weakness was easiest to find? Why?
 - The easiest weakness to find was missing backend validation. This is because we can easily test it by sending wrong data like empty name or negative age using Postman, and we can immediately see whether the system accepts it or not.
2. Which weakness was most dangerous? Why?
 - The most dangerous weakness is SQL Injection. This is because it can affect the database directly. If it is not protected, attackers can bypass login or change/delete data in the system.
3. Why is frontend validation not enough?
 - Frontend validation is not enough because users can bypass it easily using Thunder Client or browser tools. So even if the frontend blocks invalid input, the backend must still check it.
4. Why should BMI calculation be performed at the backend?
 - BMI calculation should be done at the backend because users can manipulate the frontend. If BMI is calculated in frontend, users can change the value before sending it to the server.
5. What is the difference between authentication and authorization?
 - Authentication is to check who the user is (login process).
 - Authorization is to check what the user is allowed to do after login.
6. What is the difference between RBAC and owner-based access control?
 - RBAC is based on roles like user, staff, and admin.
 - Owner-based access control means users can only access their own data, not other users' data.
7. Why is JWT alone not enough to protect all data?
 - JWT only proves the user is logged in, but it does not control everything. We still need validation, role checking, and ownership checking to fully secure the system.
8. Why should API responses exclude password_hash?
 - Because password_hash is sensitive information. Even if it is hashed, it should not be exposed in API responses to avoid security risks.
9. Why is Vue route guard not enough to protect backend routes?
 - Vue route guard only protects the frontend page. Users can still call the backend API directly using Postman, so backend protection is still needed.
10. How did you prove that your security fix works?

- We tested the system before and after fixing. After the fix, invalid data is rejected, SQL injection does not work, unauthorized users get access denied, and sensitive data like password_hash is no longer shown.